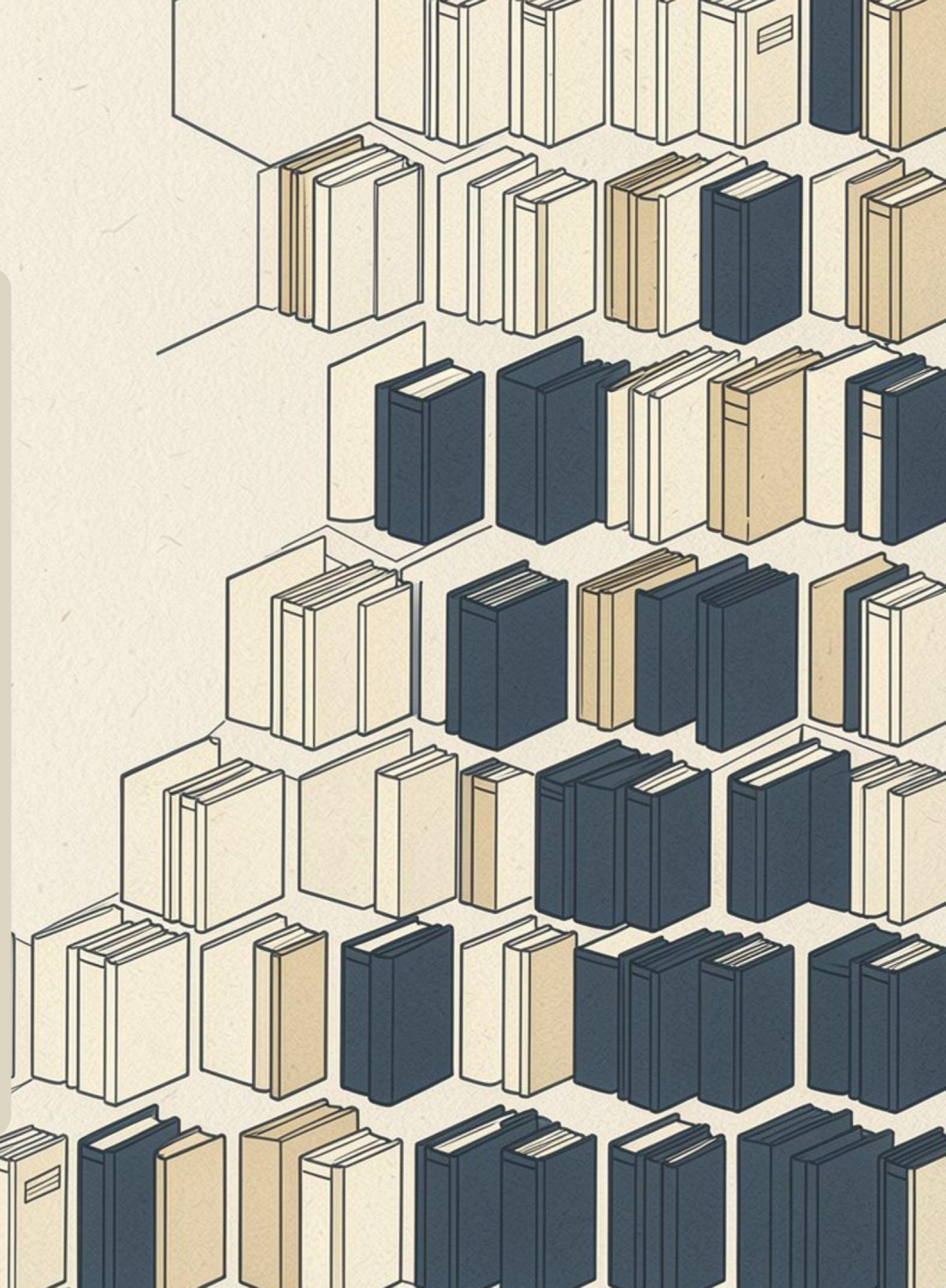
CLOUD COMPUTING · TECHNICAL SEMINAR

Serverless Computing: Reducing Infrastructure Management in the Cloud

Understanding Function-as-a-Service, architecture, and business benefits

By:
Manoj M (192511060)

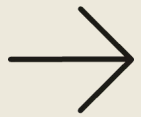


THE SHIFT

Cloud delivery shifts effort from servers to application logic

Server-based deployment

1. Provision capacity and operating systems.
2. Patch, scale, monitor, and maintain servers.
3. Carry infrastructure decisions alongside application work.



Serverless delivery

1. Request compute, storage, and networking on demand.
2. The provider handles much of the infrastructure work.
3. Developers focus more on application behavior.

Objective: understand how Function-as-a-Service reduces infrastructure management.



CLEARING UP THE TERM

Serverless still uses servers, but the provider operates them

- **Automatic scaling:** capacity responds to incoming demand.
- **Event-driven execution:** requests, uploads, messages, and schedules can invoke code.
- **Usage-based billing:** generally follows requests and execution resources; details vary by provider.

FaaS runs individual functions; **BaaS** supplies managed backend capabilities such as authentication or databases.

Examples: AWS Lambda, Azure Functions, and Google Cloud Run functions.

FUNCTION-AS-A-SERVICE

FaaS packages business logic as small, independent functions



Event

HTTP request, upload, message, or schedule arrives.



Function

Focused logic runs with defined inputs and outputs.



Result

Data changes, a response returns, or another event follows.

Infrastructure operations
Runtime and execution environment are managed by the provider. Billing commonly reflects requests and execution resources, depending on provider and configuration.

Application logic
Developers define the function's behavior and connected services. Functions scale independently as demand changes. Examples: resize an uploaded image or send a welcome email after registration.

REFERENCE ARCHITECTURE

A serverless request connects managed entry points, functions, and data

Managed cloud boundary · exact architectures vary by application

1	2	3	4	5
Client	API Gateway	Event trigger	FaaS function	Data services
Application request	Receives and routes	Creates invocation	Runs logic	Database or object storage

Monitoring and logging observe execution signals across the managed environment.

Records and files persist in managed data services.

Execution is a short lifecycle from event to observed result

01

Event generation

A request, upload, message, schedule, or data change occurs.

02

Invocation

The platform routes the event to the function.

03

Initialization

A new environment may start, or a warm one may be reused.

04

Authorized access

The function reads or writes connected services.

05

Response

Output, status, or a follow-on event is produced.

06

Signals

The provider records resource use and operational signals.

Cold start · initialization latency on a new environment.

Warm invocation · reuse of an already initialized environment.

A BALANCED VIEW

Serverless reduces operations, but trade-offs remain

Advantages	Limitations
• Less provisioning and maintenance; less infrastructure administration. • Automatic scaling for variable workloads. • Faster deployment cycles. • Pay-per-use can fit intermittent demand.	• Cold-start latency in some situations. • Execution and resource limits. • Debugging and monitoring complexity. • Vendor lock-in and provider-specific configuration. • Costs can rise for high or continuous workloads.

The fit depends on workload characteristics. Serverless is not automatically cheaper or faster.

ILLUSTRATIVE USE CASE

FaaS processes e-commerce images on demand

1	2	3	4	5
Upload	Storage event	Function	Processed storage	Website
Customer adds a product image.	Object arrival triggers work.	Validate and resize the file.	Save catalog-ready versions.	Catalog consumes the images.

Why FaaS fits: each image is independent, uploads may be irregular, and parallel work can scale with demand.

Illustrative implementation: AWS Lambda with Amazon S3.

PATTERNS BEYOND ONE USE CASE

Event-driven patterns appear across many applications



File processing

Trigger: upload
Function: validate, transform, or classify
Benefit: automation without an always-on worker



Email and notifications

Trigger: registration or order
Function: compose and send
Benefit: isolated delivery logic



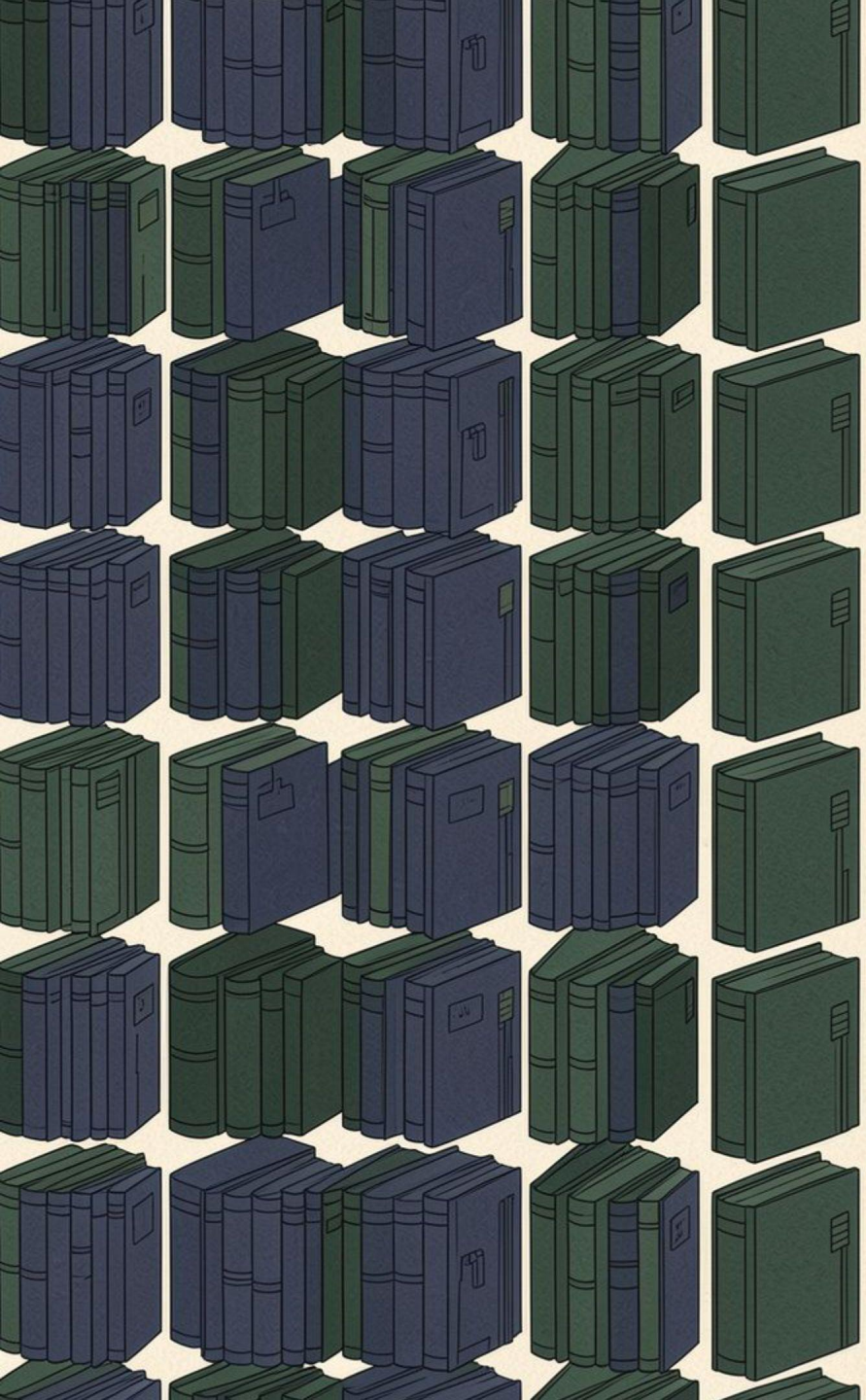
IoT processing

Trigger: device event
Function: filter, enrich, or route
Benefit: elastic handling of bursts



Scheduled reports

Trigger: timer event
Function: query and transform
Benefit: repeatable jobs without server scheduling



KEY TAKEAWAY

Use serverless where the workload rewards managed execution.

Serverless removes much of the infrastructure-management burden, not the servers. FaaS runs individual functions in response to events; automatic scaling and usage-based pricing can benefit variable workloads.

Assess latency, execution limits, security, observability, provider dependence, and cost before adoption. Serverless computing lets developers focus on application logic rather than infrastructure operations.

Thank You

References: AWS Lambda Functions documentation · Microsoft Azure Functions Overview · Google Cloud Run functions documentation